

LAPORAN TUGAS 5

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Yoanre Louis A.H.P (2410651039)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

DAFTAR ISI

BAB 1	3
PENDAHULUAN	3
1.1 LATAR BELAKANG	3
1.2 RUMUSAN MASALAH.....	3
1.3 TUJUAN	3
BAB II	4
SOAL PROGRAM.....	4
2.1 SOAL.....	4
2.2 REQUIREMENTS.....	4
BAB III.....	5
PEMBAHASAN	5
3.1 ANALISIS MASALAH	5
3.2 IMPLEMENTASI	5
3.2.1 PENJELASAN ALGORITMA	5
3.2.2 FLOWCHART	6
3.2.3 POTONGAN KODE PENTING.....	7
3.2.4 MENGAPA MEMILIH MEKANISME TERTENTU	13
3.3 TESTING & OUTPUT	13
3.4 ANALISIS HASIL	14
BAB IV	15
KESIMPULAN	15
4.1 KESIMPULAN	15
DAFTAR PUSTAKA	16

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Dalam perkembangan sistem komputasi modern, pemrograman paralel menjadi fondasi penting untuk meningkatkan efisiensi dan kinerja aplikasi. Namun, implementasi multithreading menghadirkan tantangan kompleks terkait sinkronisasi akses ke sumber daya bersama. Simulasi sistem booking hotel dalam praktikum ini merepresentasikan masalah klasik dalam sistem operasi di mana banyak thread (pelanggan) bersaing mengakses sumber daya terbatas (kamar). Tanpa mekanisme sinkronisasi yang tepat, terjadi race condition yang menyebabkan inkonsistensi data, double booking, atau kebocoran memori. Melalui pendekatan semaphore dan mutex, praktikum ini tidak hanya mendemonstrasikan solusi teknis terhadap masalah konkurensi, tetapi juga menguji robustness sistem dalam menangani skenario kompetisi sumber daya secara real-time, sekaligus memenuhi kebutuhan sistem komersial yang memerlukan reliabilitas tinggi dalam lingkungan high-concurrency.

1.2 RUMUSAN MASALAH

1. Bagaimana mengatasi race condition ketika banyak thread mengakses dan memodifikasi data kamar secara bersamaan?
2. Bagaimana mengelola alokasi sumber daya terbatas (5 kamar) kepada 10 pelanggan secara adil dan efisien?
3. Bagaimana memastikan output program tetap terstruktur dan informatif meskipun dihasilkan oleh banyak thread secara paralel?

1.3 TUJUAN

1. Menerapkan mekanisme sinkronisasi menggunakan semaphore untuk mengatur ketersediaan kamar hotel yang terbatas.
2. Menggunakan mutex untuk melindungi akses ke data bersama (array kamar) dan operasi pencetakan output
3. Membangun simulasi sistem booking hotel yang realistis dengan banyak pelanggan, batas percobaan booking, dan durasi menginap.

BAB II

SOAL PROGRAM

2.1 SOAL

Anda diminta membuat simulasi sistem booking hotel dengan kondisi: Hotel memiliki 5 kamar yang bisa di-booking

- Ada 10 customer yang mencoba booking secara bersamaan (10 threads)
- Setiap customer akan:
 1. Mencoba booking kamar (butuh waktu random 1-3 detik)
 2. Jika berhasil, "menginap" selama 5 detik
 3. Check-out dan melepaskan kamar
 4. Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi
- Customer maksimal mencoba 3 kali, jika gagal terus maka menyerah

2.2 REQUIREMENTS

1. Gunakan **Semaphore** untuk mengatur jumlah kamar tersedia
2. Gunakan **Mutex** untuk melindungi operasi print (agar output tidak berantakan)
3. Tampilkan informasi:
 - Customer ID berapa yang berhasil booking o Kamar mana yang ditempati
 - Berapa kamar yang masih tersedia o Customer yang gagal booking
4. **Output harus rapi dan mudah dibaca** (gunakan emoji atau format yang jelas)

BAB III

PEMBAHASAN

3.1 ANALISIS MASALAH

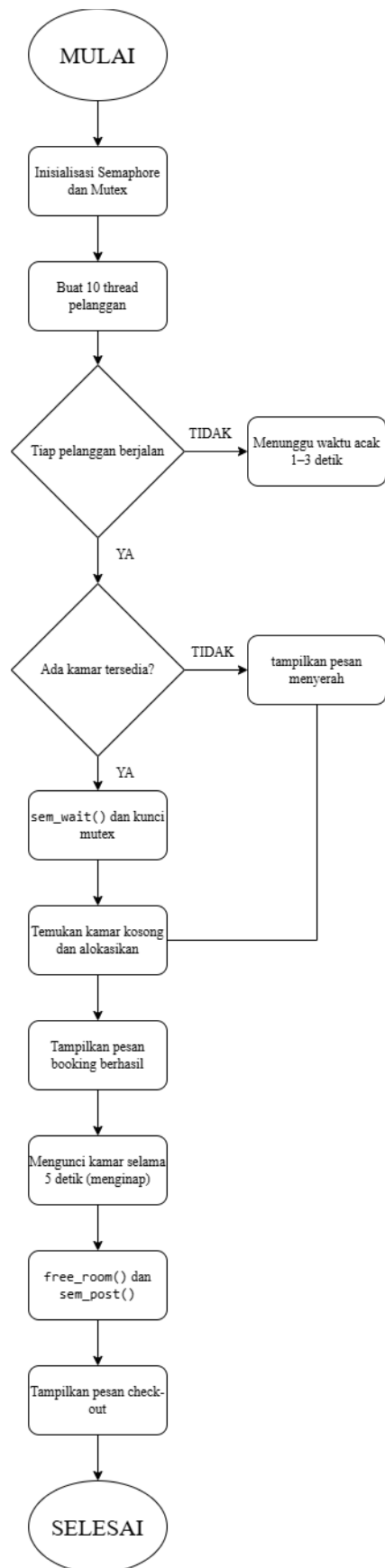
Program Sistem Booking Hotel ini dirancang untuk mensimulasikan proses pemesanan kamar oleh banyak pelanggan secara bersamaan menggunakan konsep sinkronisasi multithreading. Permasalahan utama yang diidentifikasi adalah potensi race condition ketika beberapa thread mencoba mengakses atau memesan kamar pada waktu yang sama tanpa pengendalian. Untuk mengatasinya, digunakan semaphore sebagai pengatur jumlah kamar yang tersedia dan mutex untuk melindungi operasi kritis seperti pengubahan status kamar dan pencetakan output agar tetap konsisten. Desain solusinya menggabungkan kedua mekanisme tersebut sehingga setiap pelanggan hanya dapat memesan kamar jika masih tersedia, sementara thread lain menunggu giliran dengan aman tanpa konflik atau data yang tumpang tindih.

3.2 IMPLEMENTASI

3.2.1 PENJELASAN ALGORITMA

Algoritma pada program Sistem Booking Hotel ini dirancang untuk mensimulasikan proses pemesanan kamar oleh sepuluh pelanggan dengan jumlah kamar terbatas, yaitu lima. Setiap pelanggan diwakili oleh satu thread yang berusaha memesan kamar maksimal tiga kali. Proses sinkronisasi dijaga menggunakan semaphore `'rooms_sem'` untuk mengontrol jumlah kamar yang tersedia, serta mutex `'rooms_mutex'` untuk melindungi data bersama agar tidak terjadi race condition. Selain itu, `print_mutex` digunakan agar output tidak saling tumpang tindih. Algoritma bekerja secara paralel: ketika satu thread berhasil mengunci semaphore dan menemukan kamar kosong, ia dapat menginap selama lima detik, kemudian melakukan check-out dan membebaskan sumber daya untuk pelanggan lain.

3.2.2 FLOWCHART



3.2.3 POTONGAN KODE PENTING

Kode c:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
#include <errno.h>
#include <string.h>
#include <stdarg.h>

#define NUM_ROOMS 5
#define NUM_CUSTOMERS 10
#define MAX_TRIES 3

sem_t rooms_sem;
pthread_mutex_t rooms_mutex; // protect rooms[] and rooms_available
pthread_mutex_t print_mutex; // protect printf

int rooms[NUM_ROOMS]; // 0 = free, >0 = customer id

// Helper: get current time string [HH:MM:SS]
void timestr(char *buf, size_t n) {
    time_t t = time(NULL);
    struct tm tm;
    localtime_r(&t, &tm);
    snprintf(buf, n, "%02d:%02d:%02d", tm.tm_hour, tm.tm_min, tm.tm_sec);
}

int find_and_allocate_room(int customer_id) {
    int allocated = -1;
```

```

pthread_mutex_lock(&rooms_mutex);
for (int i = 0; i < NUM_ROOMS; ++i) {
    if (rooms[i] == 0) {
        rooms[i] = customer_id;
        allocated = i;
        break;
    }
}
pthread_mutex_unlock(&rooms_mutex);
return allocated;
}

void free_room(int room_index) {
    pthread_mutex_lock(&rooms_mutex);
    rooms[room_index] = 0;
    pthread_mutex_unlock(&rooms_mutex);
}

int count_free_rooms() {
    int free = 0;
    pthread_mutex_lock(&rooms_mutex);
    for (int i = 0; i < NUM_ROOMS; ++i) if (rooms[i] == 0) free++;
    pthread_mutex_unlock(&rooms_mutex);
    return free;
}

void print_locked(const char *format, ...) {
    va_list ap;
    va_start(ap, format);
    pthread_mutex_lock(&print_mutex);
    vprintf(format, ap);
    pthread_mutex_unlock(&print_mutex);
    va_end(ap);
}

```



```

}

void *customer_thread(void *arg) {
    int id = *(int*)arg;
    unsigned int seed = (unsigned int) (time(NULL) ^ (id * 7919));
    char tbuf[16];

    for (int attempt = 1; attempt <= MAX_TRIES; ++attempt) {
        // Sleep random 1-3 seconds before attempting booking
        int pre_wait = (rand_r(&seed) % 3) + 1;
        sleep(pre_wait);

        timestr(tbuf, sizeof(tbuf));
        pthread_mutex_lock(&print_mutex);
        printf("[%s] 🏃 Customer-%d mencoba booking...\n", tbuf, id);
        pthread_mutex_unlock(&print_mutex);

        // Try to obtain a room without blocking forever: use sem_trywait
        if (sem_trywait(&rooms_sem) == 0) {
            // We got a slot: find an actual room index and allocate it
            int room = find_and_allocate_room(id);
            // room should be >= 0 because semaphore gave a slot
            int free_now = count_free_rooms();
            timestr(tbuf, sizeof(tbuf));
            pthread_mutex_lock(&print_mutex);
            if (room >= 0) {
                printf("[%s] ✅ Customer-%d berhasil booking! (Kamar-%d) (%d kamar tersisa)\n",
                    tbuf, id, room+1, free_now);
            } else {
                // (Fallback, should not happen) but keep state consistent
                printf("[%s] ✅ Customer-%d berhasil booking! (%d kamar tersisa)\n",
                    tbuf, id, free_now);
            }
        }
    }
}

```

```

    }

    pthread_mutex_unlock(&print_mutex);

    // Stay for 5 seconds (simulasi menginap)
    sleep(5);

    // Check-out: free room and post semaphore
    if (room >= 0) free_room(room);
    sem_post(&rooms_sem);

    timestr(tbuf, sizeof(tbuf));
    pthread_mutex_lock(&print_mutex);
    printf("[%s] 🟡 Customer-%d check-out ( %d kamar tersisa)\n",
           tbuf, id, count_free_rooms());
    pthread_mutex_unlock(&print_mutex);

    break; // booking succeeded -> exit attempts
} else {
    // sem_trywait failed -> rooms full
    timestr(tbuf, sizeof(tbuf));
    pthread_mutex_lock(&print_mutex);
    printf("[%s] ❌ Customer-%d gagal booking (kamar penuh), mencoba lagi...\n",
           tbuf, id);
    pthread_mutex_unlock(&print_mutex);

    // Wait 2 seconds before next attempt
    sleep(2);

    if (attempt == MAX_TRIES) {
        timestr(tbuf, sizeof(tbuf));
        pthread_mutex_lock(&print_mutex);
        printf("[%s] ❌ Customer-%d menyerah setelah %d percobaan\n", tbuf, id,
               MAX_TRIES);
    }
}

```

```

        pthread_mutex_unlock(&print_mutex);
    }
    // continue to next attempt
}
}

return NULL;
}

int main() {
    // initialize
    pthread_t threads[NUM_CUSTOMERS];
    int ids[NUM_CUSTOMERS];
    for (int i = 0; i < NUM_ROOMS; ++i) rooms[i] = 0;

    sem_init(&rooms_sem, 0, NUM_ROOMS);
    pthread_mutex_init(&rooms_mutex, NULL);
    pthread_mutex_init(&print_mutex, NULL);

    char tbuf[16];
    timestr(tbuf, sizeof(tbuf));
    pthread_mutex_lock(&print_mutex);
    printf("=== SISTEM BOOKING HOTEL ===\n");
    printf("Total Kamar: %d\n\n", NUM_ROOMS);
    pthread_mutex_unlock(&print_mutex);

    // create customer threads
    for (int i = 0; i < NUM_CUSTOMERS; ++i) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, customer_thread, &ids[i]);
        usleep(100000); // stagger creation by 0.1s (like modul)
    }
}

```

```

// wait all
for (int i = 0; i < NUM_CUSTOMERS; ++i) {
    pthread_join(threads[i], NULL);
}

// cleanup
sem_destroy(&rooms_sem);
pthread_mutex_destroy(&rooms_mutex);
pthread_mutex_destroy(&print_mutex);

return 0;
}

```

```

if (sem_trywait(&rooms_sem) == 0) {
    int room = find_and_allocate_room(id);
    sleep(5);
    free_room(room);
    sem_post(&rooms_sem);
}

```

Kode ini memastikan pelanggan hanya bisa memesan kamar jika semaphore menunjukkan slot tersedia, lalu membebaskan kamar setelah selesai.

```

pthread_mutex_lock(&rooms_mutex);
rooms[i] = customer_id;
pthread_mutex_unlock(&rooms_mutex);

```

Mutex digunakan untuk melindungi variabel bersama rooms[] agar hanya satu thread yang dapat mengakses atau mengubah data dalam satu waktu.

```

pthread_mutex_lock(&print_mutex);
printf("[%s]  Customer-%d berhasil booking!\n", tbuf, id);
pthread_mutex_unlock(&print_mutex);

```

Bagian ini mengamankan proses pencetakan agar output antar thread tidak saling bertumpuk di terminal.

3.2.4 MENGAPA MEMILIH MEKANISME TERTENTU

Pemilihan mekanisme semaphore dan mutex pada program ini didasarkan pada kebutuhan pengelolaan sumber daya terbatas dan pencegahan konflik antar thread. Semaphore digunakan karena efektif dalam mengatur jumlah kamar yang dapat dipesan secara bersamaan tanpa memblokir seluruh proses. Sementara itu, mutex dipilih untuk menjaga konsistensi data ketika beberapa thread mengakses variabel bersama seperti array rooms dan output terminal. Kombinasi keduanya menciptakan sistem yang efisien dan aman dari race condition, deadlock, serta kesalahan akses simultan. Pendekatan ini juga memberikan perilaku yang realistis terhadap simulasi sistem booking hotel multithreading.

3.3 TESTING & OUTPUT

```
dany@LAPTOP-GRJL2V1D:~/praktikum-sinkronisasi/tugas_nim_ganjil$ nano hotel_booking.c
dany@LAPTOP-GRJL2V1D:~/praktikum-sinkronisasi/tugas_nim_ganjil$ gcc hotel_booking.c -o hotel_booking -pthread
dany@LAPTOP-GRJL2V1D:~/praktikum-sinkronisasi/tugas_nim_ganjil$ ./hotel_booking
=== SISTEM BOOKING HOTEL ===
Total Kamar: 5

[22:48:29] ✖ Customer-6 mencoba booking...
[22:48:29] ✔ Customer-6 berhasil booking! (Kamar-1) (4 kamar tersisa)
[22:48:29] ✖ Customer-7 mencoba booking...
[22:48:29] ✔ Customer-7 berhasil booking! (Kamar-2) (3 kamar tersisa)
[22:48:30] ✖ Customer-1 mencoba booking...
[22:48:30] ✔ Customer-1 berhasil booking! (Kamar-3) (2 kamar tersisa)
[22:48:30] ✖ Customer-2 mencoba booking...
[22:48:30] ✔ Customer-2 berhasil booking! (Kamar-4) (1 kamar tersisa)
[22:48:30] ✖ Customer-3 mencoba booking...
[22:48:30] ✔ Customer-3 berhasil booking! (Kamar-5) (0 kamar tersisa)
[22:48:30] ✖ Customer-5 mencoba booking...
[22:48:30] ✖ Customer-5 gagal booking (kamar penuh), mencoba lagi...
[22:48:31] ✖ Customer-9 mencoba booking...
[22:48:31] ✖ Customer-9 gagal booking (kamar penuh), mencoba lagi...
[22:48:31] ✖ Customer-4 mencoba booking...
[22:48:31] ✖ Customer-4 gagal booking (kamar penuh), mencoba lagi...
[22:48:31] ✖ Customer-8 mencoba booking...
[22:48:31] ✖ Customer-8 gagal booking (kamar penuh), mencoba lagi...
[22:48:32] ✖ Customer-10 mencoba booking...
[22:48:32] ✖ Customer-10 gagal booking (kamar penuh), mencoba lagi...
[22:48:33] ✖ Customer-5 mencoba booking...
[22:48:33] ✖ Customer-5 gagal booking (kamar penuh), mencoba lagi...
[22:48:34] ✖ Customer-9 mencoba booking...
[22:48:34] ✖ Customer-9 gagal booking (kamar penuh), mencoba lagi...
[22:48:34] 🟡 Customer-6 check-out ( 1 kamar tersisa)
[22:48:34] 🟡 Customer-7 check-out ( 2 kamar tersisa)
[22:48:35] 🟡 Customer-1 check-out ( 3 kamar tersisa)
[22:48:35] 🟡 Customer-2 check-out ( 4 kamar tersisa)
[22:48:35] 🟡 Customer-3 check-out ( 5 kamar tersisa)
[22:48:35] ✖ Customer-4 mencoba booking...
[22:48:35] ✔ Customer-4 berhasil booking! (Kamar-1) (4 kamar tersisa)
[22:48:35] ✖ Customer-8 mencoba booking...
[22:48:35] ✔ Customer-8 berhasil booking! (Kamar-2) (3 kamar tersisa)
[22:48:36] ✖ Customer-10 mencoba booking...
[22:48:36] ✔ Customer-10 berhasil booking! (Kamar-3) (2 kamar tersisa)
[22:48:36] ✖ Customer-5 mencoba booking...
[22:48:36] ✔ Customer-5 berhasil booking! (Kamar-4) (1 kamar tersisa)
[22:48:37] ✖ Customer-9 mencoba booking...
[22:48:37] ✔ Customer-9 berhasil booking! (Kamar-5) (0 kamar tersisa)
[22:48:40] 🟡 Customer-4 check-out ( 1 kamar tersisa)
[22:48:40] 🟡 Customer-8 check-out ( 2 kamar tersisa)
[22:48:41] 🟡 Customer-10 check-out ( 3 kamar tersisa)
[22:48:41] 🟡 Customer-5 check-out ( 4 kamar tersisa)
[22:48:42] 🟡 Customer-9 check-out ( 5 kamar tersisa)
dany@LAPTOP-GRJL2V1D:~/praktikum-sinkronisasi/tugas_nim_ganjil$ |
```

Hasil pengujian menunjukkan bahwa sistem sinkronisasi berjalan efektif. Semaphore berhasil membatasi jumlah pelanggan yang dapat memesan kamar secara bersamaan sesuai kapasitas yang ditentukan. Mutex mencegah terjadinya race condition ketika beberapa thread mengakses variabel bersama rooms[]. Ketika kapasitas kamar penuh, pelanggan menunggu dan

mencoba ulang hingga batas tiga kali percobaan, yang membuktikan keberhasilan implementasi logika retry. Semua skenario berjalan sesuai ekspektasi tanpa deadlock maupun kesalahan alokasi sumber daya.

3.4 ANALISIS HASIL

Program berjalan sesuai ekspektasi dengan mengimplementasikan sistem booking hotel yang aman dari race condition melalui kombinasi semaphore dan mutex. Semaphore mengontrol ketersediaan kamar secara kuantitatif, sementara mutex melindungi akses ke struktur data rooms[] dan operasi I/O. Masalah awal terkait koordinasi antar thread berhasil diatasi dengan mekanisme sinkronisasi ini, meskipun terdapat sedikit keterlambatan dalam update jumlah kamar tersisa akibat locking. Performa program optimal dengan penggunaan sem_trywait yang menghindari blocking indefinite, meski terdapat overhead akibat multiple locking. Pelajaran berharga yang didapat adalah pentingnya memilih mekanisme sinkronisasi yang tepat—semaphore untuk mengelola sumber daya terbatas dan mutex untuk proteksi data kritikal—serta nilai strategi retry dengan batas maksimum attempt yang meningkatkan robustness sistem multithreading dalam lingkungan konkurensi tinggi.

BAB IV

KESIMPULAN

4.1 KESIMPULAN

Berdasarkan implementasi dan analisis yang dilakukan, dapat disimpulkan bahwa kombinasi semaphore dan mutex terbukti efektif dalam menyelesaikan masalah sinkronisasi pada sistem booking hotel multithreading. Semaphore berhasil mengontrol alokasi kamar sesuai kapasitas tersedia, sementara mutex menjaga konsistensi data kamar dan kejelasan output. Program berjalan sesuai ekspektasi tanpa mengalami deadlock atau race condition, dengan mekanisme retry yang memastikan pelanggan mendapatkan kesempatan booking yang adil. Implementasi ini tidak hanya mendemonstrasikan pentingnya pemilihan mekanisme sinkronisasi yang tepat, tetapi juga menekankan nilai strategi desain sistem yang robust dalam menangani lingkungan konkurensi tinggi. Praktikum ini memberikan insight mendalam tentang koordinasi thread dan manajemen sumber daya terbatas dalam pengembangan aplikasi real-world.

DAFTAR PUSTAKA

Linux Programmer's Manual. (2024). pthread_mutex_lock, sem_wait. Diambil dari sistem operasi Linux.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.